МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе № 3
по дисциплине
«Управление данными»
«Пользовательские функции. Процедурный язык PL/pgSQL»

Выполнил студент гр. ИВТб-2303-06-00 _____ /Перевозчиков Д.А./
Проверил _____ /Клюкин В. Л./

Киров
2026

## 1 Цели лабораторной работы

- познакомиться с созданием пользовательских функций и триггеров в PostgreSQL;
- освоить работу с составными типами данных и массивами;
- изучить основы процедурного языка PL/pgSQL.

## 2 Задание

Для базы данных, созданной в лабораторных работах № 1 и № 2, требуется:

1. для любой таблицы создать функцию `save_имя_таблицы`, которая выполняет `INSERT`, если переданный первичный ключ равен `NULL`, и `UPDATE` в противном случае, и возвращает первичный ключ обработанной строки;
2. для таблицы, на которую имеются внешние ключи, создать функцию `delete_имя_таблицы`, которая удаляет строку по первичному ключу, а при наличии ссылок на неё поднимает ошибку «Невозможно выполнить удаление, так как есть внешние ссылки»;
3. для таблицы с числовым столбцом создать функцию, возвращающую `SETOF` строк, в которых значение не меньше переданного аргумента;
4. создать составной тип из нескольких полей, хотя бы одно из которых числовое, и функцию, фильтрующую массив объектов этого типа по минимальному значению поля;
5. создать таблицу `log_имя_таблицы` для журнала изменений выбранного столбца и заполнять её триггером на вставку и изменение строк;
6. реализовать функцию, использующую динамически формируемый запрос.

Все функции реализуются на PL/pgSQL. Работа выполняется на базе данных гоночного автоклуба (таблицы `car`, `trainer`, `trainer_car`, `competition`, `achievement`) в PostgreSQL 16. Основным объектом заданий выбрана таблица автомобилей `car`: на неё ссылаются две таблицы, и она содержит числовые столбцы `horsepower` и `production_year`. Состояние таблицы перед выполнением работы приведено в таблице 1. Пропуск ключа 12 и три одинаковые строки Volkswagen Beetle (13–15) остались после повторных запусков запроса `INSERT` в лабораторной работе № 2.2.

Таблица 1 – Исходное содержимое таблицы car (основные столбцы)

| **id** | **title** | **horsepower** | **production_year** | **color** | **is_utilized** |
|---|---|---|---|---|---|
| 1 | ВАЗ 2107 | 76 | 1995 | Белый | false |
| 2 | OKA | 200 | 2000 | Красный | true |
| 3 | Mazda MX-5 Miata | 221 | 2022 | Красный | false |
| 4 | Toyota Supra | 330 | 1997 | Оранжевый | false |
| 5 | Nissan Skyline GT-R | 280 | 1999 | Синий | false |
| 6 | Mitsubishi Lancer Evolution | 180 | 2005 | Красный | false |
| 7 | BMW M3 | 510 | 2021 | Серый | false |
| 8 | Audi Quattro | 200 | 1984 | Оранжевый | false |
| 9 | Peugeot 406 | 194 | 1997 | Белый | false |
| 10 | Porsche 911 | 450 | 2022 | Чёрный | false |
| 11 | Audi RS2 | 315 | 1994 | Синий | false |
| 13 | Volkswagen Beetle | 111 | 1975 | Красный | false |
| 14 | Volkswagen Beetle | 111 | 1975 | Красный | false |
| 15 | Volkswagen Beetle | 111 | 1975 | Красный | false |

Запросы каждого задания сохранены в отдельном файле каталога `sql`: файл содержит создание объектов и проверочные запросы, которые приведены ниже по ходу описания.

# 3 Решение

## 3.1 Функция save_car

Код задания находится в файле `save_car.sql`. Функция принимает значения всех столбцов таблицы `car`. Имена параметров оканчиваются символом подчёркивания, чтобы они не совпадали с именами столбцов внутри запросов. Параметру `is_utilized_` задано значение по умолчанию `FALSE`, как и одноимённому столбцу, поэтому при вызове его можно не указывать.

Если `id_` равен `NULL`, выполняется `INSERT`, и сгенерированный последовательностью ключ попадает в переменную `used_id` через `RETURNING ... INTO`. Иначе выполняется `UPDATE`. В отличие от примера из методических указаний, где после обновления в переменную просто копируется переданный ключ, здесь он тоже берётся из `RETURNING`. Поэтому при обновлении несуществующей строки функция возвращает `NULL`, и вызывающий код может отличить успешное изменение от холостого.

```
CREATE OR REPLACE FUNCTION save_car(
    id_ BIGINT,
    title_ VARCHAR(100),
    photo_url_ VARCHAR(255),
    description_ TEXT,
    horsepower_ INT,
    production_year_ INT,
    color_ VARCHAR(20),
    suspension_name_ VARCHAR(100),
    is_utilized_ BOOLEAN DEFAULT FALSE
)
RETURNS BIGINT
AS $$
DECLARE
    used_id BIGINT;
BEGIN
    IF id_ IS NULL THEN
        INSERT INTO car (title, photo_url, description, horsepower,
                         production_year, color, suspension_name, is_utilized)
        VALUES (title_, photo_url_, description_, horsepower_,
                production_year_, color_, suspension_name_, is_utilized_)
```

```
        RETURNING id INTO used_id;
    ELSE
        UPDATE car SET
            title = title_,
            photo_url = photo_url_,
            description = description_,
            horsepower = horsepower_,
            production_year = production_year_,
            color = color_,
            suspension_name = suspension_name_,
            is_utilized = is_utilized_
        WHERE id = id_
        RETURNING id INTO used_id; -- останется NULL, если строки с таким id
        ↪ нет
    END IF;

    RETURN used_id;
END;
$$ LANGUAGE plpgsql;
```

Функция проверена тремя вызовами: добавлением нового автомобиля, изменением описания и мощности автомобиля Peugeot 406 (id = 9) и попыткой изменить строку с несуществующим ключом 999.

```
-- Проверка: вставка новой строки
SELECT save_car(NULL, 'Subaru Impreza WRX STI', NULL,
                'Оппозитный турбомотор и симметричный полный привод.',
                300, 2006, 'Синий', 'MacPherson');

-- Проверка: изменение существующей строки
SELECT save_car(9, 'Peugeot 406', 'https://cars.pics/peugeot_406.jpg',
                'Такси из Марселя. 3.0 V6 после доработки, подвеска от
                ↪ 605-й.',
                220, 1997, 'Белый', 'Independent multi-link');

-- Проверка: изменение несуществующей строки
SELECT save_car(999, 'Lada Granta', NULL, NULL, 98, 2015, 'Серый',
↪ 'MacPherson');
```

Вызовы вернули значения 16, 9 и NULL соответственно. Новая строка

получила ключ 16, так как последовательность `car_id_seq` уже выдала значения до 15 включительно. Состояние затронутых строк показано в таблице 2: строка с id = 16 добавлена, у строки с id = 9 изменились описание и мощность (было 194 л.с.), а вызов с ключом 999 таблицу не изменил.

```
SELECT id, title, description, horsepower, production_year, color
FROM car
WHERE id IN (9, 16);
```

Таблица 2 – Строки таблицы car после вызовов save_car

| id | title | description | horsepower | production_year | color |
|---|---|---|---|---|---|
| 16 | Subaru Impreza WRX STI | Оппозитный турбомотор и симметричный полный привод. | 300 | 2006 | Синий |
| 9 | Peugeot 406 | Такси из Марселя. 3.0 V6 после доработки, подвеска от 605-й. | 220 | 1997 | Белый |

### 3.2 Функция delete_car

Код задания находится в файле `delete_car.sql`. На таблицу `car` ссылаются две таблицы: `trainer` (любимый автомобиль тренера, `ON DELETE SET NULL`) и `trainer_car` (автомобили, на которых тренер ведёт занятия, `ON DELETE CASCADE`). В методических указаниях ошибка формируется перехватом исключения `foreign_key_violation`, но в данной схеме это исключение не возникнет: при удалении автомобиля СУБД сама обнулит ссылку в `trainer` и удалит связанные строки `trainer_car`. Поэтому наличие ссылок проверяется явно, предикатами `EXISTS`, до выполнения `DELETE`. Проверка по `trainer_car` использует индекс `trainer_car(car_id)`, созданный в схеме; для столбца `trainer.favorite_car_id` при большом числе тренеров индекс стоит добавить.

Ошибка поднимается с кодом `foreign_key_violation`, чтобы клиентское приложение обрабатывало её так же, как нарушение внешнего ключа, а в поле `DETAIL` передаётся id автомобиля.

```
CREATE OR REPLACE FUNCTION delete_car(
```

```
    id_ BIGINT
)
RETURNS VOID
AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM trainer WHERE favorite_car_id = id_)
       OR EXISTS (SELECT 1 FROM trainer_car WHERE car_id = id_) THEN
        RAISE EXCEPTION 'Невозможно выполнить удаление, так как есть внешние
        ↪  ссылки'
            USING ERRCODE = 'foreign_key_violation',
                  DETAIL = format('На автомобиль с id = %s ссылаются таблицы
                   ↪  trainer или trainer_car', id_);
    END IF;

    DELETE FROM car WHERE id = id_;
END;
$$ LANGUAGE plpgsql;
```

Для проверки выбраны два автомобиля. Строка с id = 15 — один из дубликатов Volkswagen Beetle, она не закреплена ни за одним тренером, и её удаление заодно убирает лишнюю запись. Toyota Supra (id = 4) указана в `trainer_car` у тренеров Торрето и Смирнова.

```
-- Автомобиль 15 (дубликат Volkswagen Beetle) не закреплён ни за одним
↪  тренером
SELECT delete_car(15);

-- На автомобиль 4 (Toyota Supra) ссылается trainer_car
SELECT delete_car(4);
```

Первый вызов завершился без ошибок. Второй был прерван с сообщением:

```
ERROR:  Невозможно выполнить удаление, так как есть внешние ссылки
DETAIL:  На автомобиль с id = 4 ссылаются таблицы trainer или trainer_car
CONTEXT:  PL/pgSQL function delete_car(bigint) line 5 at RAISE
```

Контрольный запрос возвращает одну строку — `(4, Toyota Supra)`: автомобиль с id = 15 удалён, автомобиль с id = 4 остался в таблице вместе со ссылками на него.

```sql
SELECT id, title FROM car WHERE id IN (4, 15);
```

### 3.3 Функция фильтрации автомобилей по мощности

Код задания находится в файле `filter_car_by_horsepower.sql`. Функция `filter_car_by_horsepower` возвращает `SETOF car`, то есть множество строк той же структуры, что и таблица. Результат запроса передаётся вызывающему через `RETURN QUERY`. Строки упорядочены по убыванию мощности.

```sql
CREATE OR REPLACE FUNCTION filter_car_by_horsepower(
    min_horsepower_ INT
)
RETURNS SETOF car
AS $$
BEGIN
    RETURN QUERY
        SELECT *
        FROM car
        WHERE horsepower >= min_horsepower_
        ORDER BY horsepower DESC;
END;
$$ LANGUAGE plpgsql;
```

Функция, возвращающая множество строк, вызывается в предложении `FROM`, как таблица, поэтому из её результата можно выбирать отдельные столбцы:

```sql
SELECT id, title, horsepower, production_year, color
FROM filter_car_by_horsepower(300);
```

Таблица 3 – Результат filter_car_by_horsepower(300)

| id | title | horsepower | production_year | color |
|---|---|---|---|---|
| 7 | BMW M3 | 510 | 2021 | Серый |
| 10 | Porsche 911 | 450 | 2022 | Чёрный |
| 4 | Toyota Supra | 330 | 1997 | Оранжевый |
| 11 | Audi RS2 | 315 | 1994 | Синий |
| 16 | Subaru Impreza WRX STI | 300 | 2006 | Синий |

В выборку вошли пять автомобилей, в том числе Subaru с мощностью ровно 300 л.с., так как используется нестрогое сравнение.

### 3.4 Составной тип t_car_brief и фильтрация массива

Код задания находится в файле `car_brief_array.sql`. Составной тип `t_car_brief` описывает краткую карточку автомобиля из четырёх полей, три из которых числовые. Типы полей совпадают с типами столбцов таблицы `car`, поэтому объекты типа можно получать прямо из таблицы.

```
CREATE TYPE t_car_brief AS (
    id BIGINT,
    title VARCHAR(100),
    horsepower INT,
    production_year INT
);
```

В методических указаниях массив фильтруется SQL-запросом через `unnest`. Здесь использован другой способ — цикл `FOREACH` по элементам массива: элемент, удовлетворяющий условию, добавляется в результирующий массив функцией `array_append`. Результат инициализирован пустым массивом `'{}'`, поэтому при отсутствии подходящих элементов функция возвращает пустой массив, а не `NULL`.

```
CREATE OR REPLACE FUNCTION filter_car_brief_array(
    arr_ t_car_brief[],
    min_horsepower_ INT
)
RETURNS t_car_brief[]
AS $$
DECLARE
    item t_car_brief;
    result t_car_brief[] := '{}';
BEGIN
    FOREACH item IN ARRAY arr_ LOOP
        IF item.horsepower >= min_horsepower_ THEN
            result := array_append(result, item);
        END IF;
```

```
    END LOOP;

    RETURN result;
END;
$$ LANGUAGE plpgsql;
```

Для проверки массив строится конструктором `ARRAY(подзапрос)` из японских автомобилей, отобранных регулярным выражением по названию: Mazda (221 л.с.), Toyota, Nissan, Mitsubishi (180 л.с.) и Subaru. Каждая строка приводится к типу `t_car_brief` оператором `::`. Минимальная мощность — 250 л.с.

```
-- Массив формируется из автомобилей японского производства
SELECT filter_car_brief_array(
    ARRAY(
        SELECT (id, title, horsepower, production_year)::t_car_brief
        FROM car
        WHERE title ~ '^(Toyota|Nissan|Mazda|Mitsubishi|Subaru)'
        ORDER BY id
    ),
    250
);
```

Функция вернула массив из трёх объектов:

```
{"(4,\"Toyota Supra\",330,1997)","(5,\"Nissan Skyline GT-R\",280,1999)","(16,\"Subaru
↪  Impreza WRX STI\",300,2006)"}
```

Mazda и Mitsubishi отброшены, так как их мощность ниже 250 л.с. Для удобного просмотра результат можно развернуть в таблицу функцией `unnest`, которая для массива составного типа возвращает отдельный столбец на каждое поле (таблица 4).

```
-- Развёртывание результата в таблицу для наглядности
SELECT *
FROM unnest(filter_car_brief_array(
    ARRAY(
        SELECT (id, title, horsepower, production_year)::t_car_brief
        FROM car
```

```
        WHERE title ~ '^(Toyota|Nissan|Mazda|Mitsubishi|Subaru)'
        ORDER BY id
    ),
    250
));
```

Таблица 4 – Отфильтрованный массив, развёрнутый функцией unnest

| id | title | horsepower | production_year |
|---|---|---|---|
| 4 | Toyota Supra | 330 | 1997 |
| 5 | Nissan Skyline GT-R | 280 | 1999 |
| 16 | Subaru Impreza WRX STI | 300 | 2006 |

### 3.5 Журнал изменений мощности log_car

Код задания находится в файле `log_car.sql`. Журналируется столбец `horsepower`: мощность автомобиля меняется при доработках, и история этих изменений нужна для учёта. Таблица `log_car` содержит первичный ключ, внешний ключ на автомобиль, момент изменения, старое и новое значение мощности. Старое значение допускает `NULL`, поскольку при вставке строки его нет. Внешний ключ объявлен с `ON DELETE CASCADE`: записи журнала не имеют смысла без автомобиля и удаляются вместе с ним.

```
CREATE TABLE IF NOT EXISTS log_car (
    id BIGSERIAL PRIMARY KEY,
    car_id BIGINT NOT NULL REFERENCES car(id) ON DELETE CASCADE,
    change_datetime TIMESTAMP NOT NULL DEFAULT NOW(),
    old_horsepower INT,
    new_horsepower INT NOT NULL
);
```

Триггерная функция определяет вид операции по переменной `TG_OP`. При `UPDATE` старое значение берётся из записи `OLD`; если мощность не изменилась (например, `UPDATE` присвоил столбцу то же значение), запись в журнал не добавляется. При `INSERT` старое значение равно `NULL`.

```
CREATE OR REPLACE FUNCTION log_car_horsepower()
RETURNS TRIGGER
```

```
AS $$
DECLARE
    old_val INT;
BEGIN
    IF TG_OP = 'UPDATE' THEN
        IF OLD.horsepower = NEW.horsepower THEN
            RETURN NEW; -- мощность не изменилась, запись в журнал не нужна
        END IF;
        old_val := OLD.horsepower;
    ELSE
        old_val := NULL;
    END IF;

    INSERT INTO log_car (car_id, old_horsepower, new_horsepower)
    VALUES (NEW.id, old_val, NEW.horsepower);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Триггер срабатывает после операции (`AFTER`), так как в журнал должны попадать только фактически выполненные изменения. Указание `UPDATE OF horsepower` ограничивает срабатывание при обновлении командами, в которых этот столбец присутствует в списке `SET`; изменение других столбцов функцию не вызывает. Вместо устаревшей формы `EXECUTE PROCEDURE` использована `EXECUTE FUNCTION`, поддерживаемая с PostgreSQL 11.

```
CREATE TRIGGER log_car_horsepower_change
AFTER INSERT OR UPDATE OF horsepower
ON car
FOR EACH ROW
EXECUTE FUNCTION log_car_horsepower();
```

Для проверки добавлен новый автомобиль, затем дважды изменена мощность (у нового автомобиля и у Mazda MX-5 Miata) и отдельно изменён цвет:

```
-- Проверка: вставка, два изменения мощности и изменение другого столбца
INSERT INTO car (title, horsepower, production_year, color, suspension_name)
```

```sql
VALUES ('Ford Focus RS', 350, 2016, 'Синий', 'MacPherson');

UPDATE car SET horsepower = 375 WHERE title = 'Ford Focus RS';

UPDATE car SET horsepower = horsepower + 30 WHERE id = 3;

UPDATE car SET color = 'Голубой' WHERE title = 'Ford Focus RS';

SELECT l.id, l.car_id, c.title, l.change_datetime,
       l.old_horsepower, l.new_horsepower
FROM log_car l
JOIN car c ON c.id = l.car_id
ORDER BY l.id;
```

Таблица 5 – Содержимое таблицы log_car

| id | car_id | title | change_datetime | old_horsepower | new_horsepower |
|---|---|---|---|---|---|
| 1 | 17 | Ford Focus RS | 2026-11-11 00:59:14.490 | null | 350 |
| 2 | 17 | Ford Focus RS | 2026-11-11 00:59:14.491 | 350 | 375 |
| 3 | 3 | Mazda MX-5 Miata | 2026-11-11 00:59:14.492 | 221 | 251 |

Вставка зафиксирована записью без старого значения, оба изменения мощности — записями со старым и новым значениями. Изменение цвета в журнал не попало, так как не затрагивает столбец `horsepower`.

### 3.6 Динамический поиск автомобилей

Код задания находится в файле `search_car.sql`. Функция `search_car` выполняет поиск по необязательным критериям: части названия, цвету, минимальной мощности и признаку исключения утилизированных автомобилей; дополнительно задаётся столбец сортировки. Текст запроса заранее неизвестен: в предложение `WHERE` попадают только условия для переданных параметров, поэтому запрос собирается в строковой переменной и выполняется командой `RETURN QUERY EXECUTE`.

Значения параметров не вставляются в текст запроса, а передаются через `USING` и подставляются вместо `$1`, `$2`, `$3`. Это исключает SQL-инъекцию

через значения и избавляет от экранирования кавычек. Имя столбца сортировки нельзя передать параметром, поэтому оно сначала сверяется со списком допустимых столбцов, а затем подставляется функцией `format` со спецификатором `%I`, который оформляет строку как идентификатор. Если сортировка выполняется не по ключу, к ней добавляется `id`: в таблице есть строки с одинаковым годом выпуска и мощностью, и без этого их порядок не определён. Сформированный запрос выводится через `RAISE NOTICE`, чтобы при проверке было видно, какой текст выполняется.

```
CREATE OR REPLACE FUNCTION search_car(
    title_ VARCHAR DEFAULT NULL,
    color_ VARCHAR DEFAULT NULL,
    min_horsepower_ INT DEFAULT NULL,
    only_active_ BOOLEAN DEFAULT TRUE,
    order_by_ TEXT DEFAULT 'id'
)
RETURNS SETOF car
AS $$
DECLARE
    query TEXT := 'SELECT * FROM car WHERE TRUE';
BEGIN
    IF title_ IS NOT NULL THEN
        query := query || ' AND title ILIKE ''%'' || $1 || ''%''';
    END IF;
    IF color_ IS NOT NULL THEN
        query := query || ' AND color = $2';
    END IF;
    IF min_horsepower_ IS NOT NULL THEN
        query := query || ' AND horsepower >= $3';
    END IF;
    IF only_active_ THEN
        query := query || ' AND NOT is_utilized';
    END IF;

    IF order_by_ NOT IN ('id', 'title', 'horsepower', 'production_year') THEN
        RAISE EXCEPTION 'Недопустимый столбец для сортировки: %', order_by_;
    END IF;
    query := query || format(' ORDER BY %I', order_by_);
```

```
    IF order_by_ <> 'id' THEN
        query := query || ', id'; -- однозначный порядок строк с равными
        ↪ значениями
    END IF;

    RAISE NOTICE '%', query;

    RETURN QUERY EXECUTE query USING title_, color_, min_horsepower_;
END;
$$ LANGUAGE plpgsql;
```

Начальное условие `WHERE TRUE` позволяет присоединять любое условие через `AND`, не проверяя, первое ли оно. Все параметры имеют значения по умолчанию, поэтому при вызове указываются только нужные, по именам (нотация `=>`).

**Вызов 1.** Синие автомобили, отсортированные по мощности.

```
-- Синие автомобили, отсортированные по мощности
SELECT id, title, color, horsepower
FROM search_car(color_ => 'Синий', order_by_ => 'horsepower');
```

```
NOTICE:  SELECT * FROM car WHERE TRUE AND color = $2 AND NOT is_utilized ORDER BY
↪ horsepower, id
```

Таблица 6 – Результат вызова 1

| id | title | color | horsepower |
|---|---|---|---|
| 5 | Nissan Skyline GT-R | Синий | 280 |
| 16 | Subaru Impreza WRX STI | Синий | 300 |
| 11 | Audi RS2 | Синий | 315 |

**Вызов 2.** Автомобили, в названии которых есть «audi» (без учёта регистра), мощностью от 250 л.с.

```
-- Автомобили Audi мощностью от 250 л.с.
SELECT id, title, color, horsepower
FROM search_car(title_ => 'audi', min_horsepower_ => 250);
```

```
NOTICE:  SELECT * FROM car WHERE TRUE AND title ILIKE '%' || $1 || '%' AND horsepower >= $3
↪ AND NOT is_utilized ORDER BY id
```

Таблица 7 – Результат вызова 2

| id | title | color | horsepower |
|---|---|---|---|
| 11 | Audi RS2 | Синий | 315 |

Audi Quattro (200 л.с.) отсеяна условием на мощность.

**Вызов 3.** Все автомобили, включая утилизированные, по году выпуска.

```
-- Все автомобили, включая утилизированные, по году выпуска
SELECT id, title, production_year, is_utilized
FROM search_car(only_active_ => FALSE, order_by_ => 'production_year');

NOTICE:  SELECT * FROM car WHERE TRUE ORDER BY production_year, id
```

Таблица 8 – Результат вызова 3

| id | title | production_year | is_utilized |
|---|---|---|---|
| 13 | Volkswagen Beetle | 1975 | false |
| 14 | Volkswagen Beetle | 1975 | false |
| 8 | Audi Quattro | 1984 | false |
| 11 | Audi RS2 | 1994 | false |
| 1 | ВАЗ 2107 | 1995 | false |
| 4 | Toyota Supra | 1997 | false |
| 9 | Peugeot 406 | 1997 | false |
| 5 | Nissan Skyline GT-R | 1999 | false |
| 2 | OKA | 2000 | true |
| 6 | Mitsubishi Lancer Evolution | 2005 | false |
| 16 | Subaru Impreza WRX STI | 2006 | false |
| 17 | Ford Focus RS | 2016 | false |
| 7 | BMW M3 | 2021 | false |
| 3 | Mazda MX-5 Miata | 2022 | false |
| 10 | Porsche 911 | 2022 | false |

Ни один фильтр не задан, поэтому запрос состоит только из начального условия и сортировки. В результат попала утилизированная OKA, а строка 15, удалённая в задании 2, отсутствует. Автомобили одного года (Toyota Supra и Peugeot 406, Mazda MX-5 Miata и Porsche 911) упорядочены по ключу.

## Вывод

В ходе лабораторной работы были реализованы пользовательские функции для таблицы `car` базы данных гоночного автоклуба, триггеры для автоматического логирования изменений, а также функция для выполнения динамических SQL-запросов.

Получены и закреплены практические навыки проектирования бизнес-логики на процедурном языке PL/pgSQL, работе с исключениями и управления целостностью базы данных.